Data Pipelines with Apache Airflow 3.2

By, Bhavani Ravi

thelearning.dev

Setup

Data Pipelines with Apache Airflow 3.2

Clone or fork the repo, follow the Readme.md instructions

https://github.com/thelearningdev/pyconus-2026-apache-airflow-tutorial

or

https://bit.ly/pycon2026-airflow-tutorial

Airflow

Data Engineering

The Bookshop

WHAT THE BUSINESS WANTS

- Know which genres sell best each day
- Confidence that reruns do not duplicate rows
- Bad sales records flagged, not silently loaded
- Daily report ready without manual intervention

WHAT THE TEAM ACTUALLY HAS

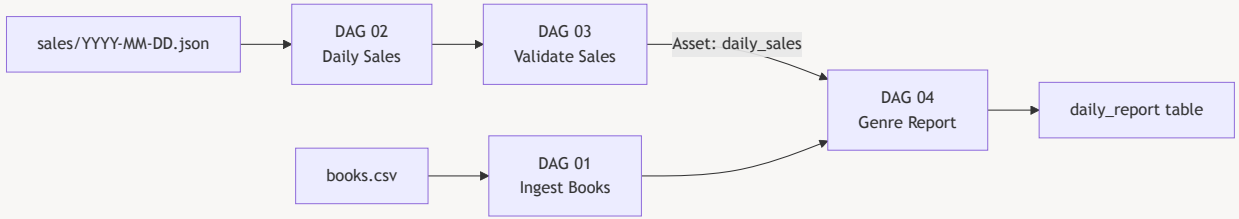
- A 25-row books catalog CSV with a few bad rows
- Daily JSON sales files that contain invalid records
- Scripts that are not scheduled and not observable
- No way to tell which day’s data has already been loaded

In today's Session

1. Introduction to Apache Airflow
2. Data Ingestion & Idempotency
3. Backfill, incremental load and scheduling
4. Data validation and quarantine
5. Build Report

The Final Pipeline

At the end of this workshop we will have...



Setup

Clone or fork the repo, follow the Readme.md instructions

https://github.com/thelearningdev/pyconus-2026-apache-airflow-tutorial

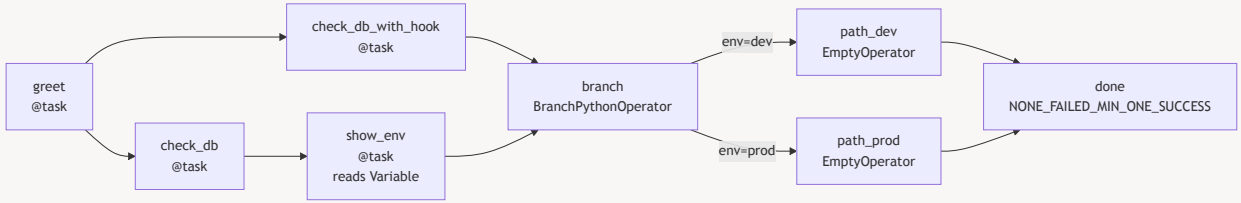
Apache Airflow 101

Hello World

Core Airflow Concepts in One DAG

Before the real pipeline, build one DAG that exercises every concept you will rely on.

The DAG — What We Are Building



Exercise 0

Run the Hello World DAG

Trigger it, break it with a missing Variable, fix it, and watch which branch runs.

`dags/00_hello_world.py`

The DAG

```python
@dag(
    dag_id="00_hello_world",
    start_date=datetime(2026, 1, 1),
    schedule=None,       # manual trigger only
    catchup=False,
    tags=["workshop", "hello-world"],
)
def hello_world():
    ...

hello_world()            # registers the DAG with Airflow
```

- A DAG is a Python function decorated with `@dag`
- `schedule=None` means the DAG only runs when you click **Trigger DAG**
- `catchup=False` prevents Airflow from backfilling historical runs on first load
- The function call at the bottom instantiates and registers the DAG object

Tasks and Operators

`@task` — Python shorthand

```python
@task
def say_hello() -> str:
    message = "Hello, Airflow 3!"
    print(message)
    return message
```

- Wraps a plain Python function as a task
- Return value is automatically stored as XCom
- Cleaner than instantiating `PythonOperator` directly

`BashOperator` — run shell commands

```python
check_date = BashOperator(
    task_id="check_date",
    bash_command="echo 'Today: '
$(date)",
)
```

- Use when the work is a shell command, not Python
- `task_id` is how Airflow identifies the task in the UI and logs

XCom (Cross-Task Communication)

```python
@task
def say_hello() -> str:
    return "Hello, Airflow 3!"          # pushed to XCom automatically

@task
def log_greeting(greeting: str) -> None:
    print(f"Received: {greeting!r}")    # pulled from XCom automatically

greeting = say_hello()
log_greeting(greeting)                  # wire the return value directly
```

- XCom lets tasks share small values (strings, dicts, lists)
- With `@task`, passing a return value as an argument wires XCom transparently
- Keep XCom small — large datasets belong in storage, not in Airflow's metadata DB

Task Chaining

```
greet_task = greet()
hook_task = check_db_with_hook()

greet_task >> hook_task >> branch
greet_task >> check_db() >> show_env() >> branch
branch >> [path_dev, path_prod]
[path_dev, path_prod] >> done
```

- `>>` declares a dependency: left must complete before right starts
- Putting tasks in a list `[a, b] >> c` means both `a` and `b` must finish
- Airflow resolves the full dependency graph before scheduling any task

BranchPythonOperator

```python
def _pick_branch() -> str:
    env = Variable.get("bookshop_env", default_var="dev")
    return "path_prod" if env == "prod" else "path_dev"

branch = BranchPythonOperator(
    task_id="branch",
    python_callable=_pick_branch,
)

branch >> [path_dev, path_prod]
```

- Returns the `task_id` (or a list) of the branch to follow
- Tasks on the unchosen path are **skipped**, not failed
- Real use cases: environment checks, feature flags, data-driven routing

Trigger Rules

```
join = EmptyOperator(
    task_id="join",
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,
)
```

Default: `ALL_SUCCESS`

- Every upstream task must succeed
- A skipped branch **blocks** the downstream task
- Works fine when there is no branching

`NONE_FAILED_MIN_ONE_SUCCESS`

- No upstream task failed
- At least one upstream task succeeded
- Skipped tasks are acceptable — the join proceeds

After a branch, always set `trigger_rule` on the join task, or Airflow will wait for a success that will never arrive.

Connections

1. A named, encrypted credential stored in Airflow. Your DAG code refers to it by ID — the actual host, port, and password live in the UI, not in the code
2. To create it, Go to Airflow UI → Admin → Connections → + Add. Set Conn ID to `bookshop_postgres`, Conn Type to `Postgres`

Used when: The same DAG file works in dev and prod because the connection ID stays the same — only the credentials change per environment.

Providers

- Providers are python packages that follow airflow specs
- This enables airflow to connect, run task, pull data from an external system
- Common providers include `standard`, `postgres`, `aws`, `google`

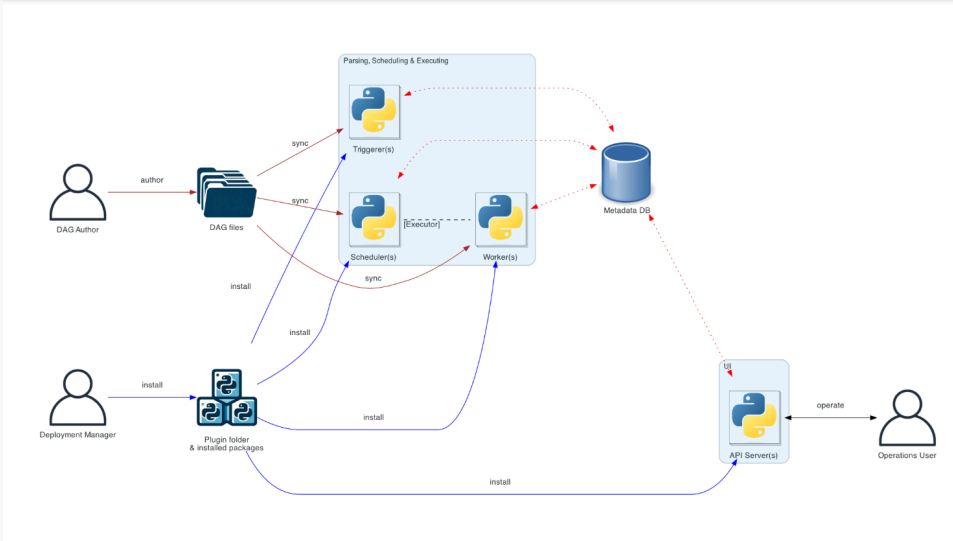
Mini-Exercise

Explore Provider Packages

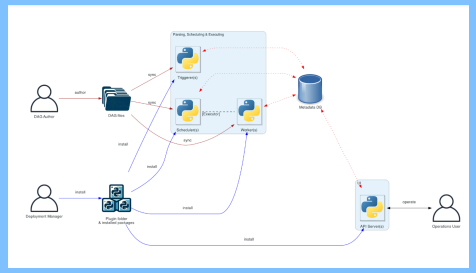
https://airflow.apache.org/docs/#providers-packages

Airflow Architecture

- Scheduler
- API Server
- Executor/Worker
- Triggerers
- Dag Processor
- Metadata DB



Airflow Architecture



- **Scheduler** — continuously parses DAG files and queues tasks that are ready to run
- **API Server** — serves the UI and the REST API; reads state from the metadata DB
- **Executor** — decides where tasks run (same process, separate process, or remote worker)
- **Triggerer** — like worker, but for deferred tasks so it doesn't block other work
- **Dag processor** — serializes DAGs and makes them available to other components
- **Metadata DB** — the source of truth for all run history, XCom, connections, and variables

Executors

*This workshop uses **SequentialExecutor** — one task at a time, zero infrastructure setup. The same DAG file runs unchanged on LocalExecutor, CeleryExecutor, or KubernetesExecutor in production.*

- **SequentialExecutor** — one task at a time; default for `airflow standalone`
- **LocalExecutor** — parallel tasks on a single machine; good for team development
- **CeleryExecutor / KubernetesExecutor** — distributed workers for production scale

The executor is an infrastructure concern, not a DAG concern. The same DAG file runs on any executor without changes.

Airflow: Reflections

- Airflow pipeline concetps - Dag, Task, XCom, Variables, Connections...
- DAG Code Concepts - Chaining, Branching, Trigger Rules
- Airflow Infrastructure

> *Airflow is still an orchestration engine, not a transformation engine*

Data Engineering

Data Engineering

MOVING DATA

Getting raw data from sources (files, APIs, databases) into a place where it can be used.

TRANSFORMING DATA

Cleaning, validating, joining, and reshaping data so it is fit for a specific question.

SERVING DATA

Making the result reliably available — to a dashboard, an API, an ML model, or another team.

DATA ENGINEERING IS THE PLUMBING

Analysts and scientists depend on it being invisible when it works and loud when it breaks.

Orchestration

SCHEDULING

Run this pipeline daily at 06:00, or whenever the upstream data arrives.

DEPENDENCY MANAGEMENT

Do not start the enrichment step until the ingest step has finished successfully.

RETRY AND ALERTING

If a step fails, retry it up to three times, then page someone.

OBSERVABILITY

Show exactly what ran, when, how long it took, and what output it produced.

AIRFLOW'S ROLE

Airflow is the orchestrator. Your Python code is the work. Airflow manages everything around it.

DagRun

- Every time we trigger a pipeline, an instance(object) of the dag is created
- This instance has the information of all tasks that needs to run in that pipeline
- It has a state success/failure
- You can clear a DagRun to run again in the same instance

Task Instances

- Similar to DagRun, TaskInstances are created for per task per dag on trigger
- TaskInstances has states like `Success,Failure,Skipped,Running`
- You can clear a specific task to rerun again

Mini-Exercise

Explore DagRun and Task Instance in UI

Module 1

Load the Data

Catalog Data

(books.csv)

ISBN	Title	Author	Genre	Price
9780743273565	The Great Gatsby	F. Scott Fitzgerald	Fiction	12.99
9780061096525	To Kill a Mockingbird	Harper Lee	Fiction	13.99
9780743477550	1984	George Orwell	Fiction	11.99

Connections (Recap)

1. A named, encrypted credential stored in Airflow.
2. Your DAG code refers to it by ID
3. The actual credentials host, port, and password live in the Airflow DB, not in your code.
4. You can also use external secret managers

> *We can use the same credential across multiple dags*

Hooks with Connection

```python
from airflow.providers.postgres.hooks.postgres import PostgresHook

hook = PostgresHook(postgres_conn_id="bookshop_postgres")

# Run a query, get results as list of tuples
rows = hook.get_records("SELECT isbn, title FROM books")

# Bulk insert rows
hook.insert_rows(
    table="books",
    rows=[("978...", "Dune", "Frank Herbert", "Sci-Fi", 16.99)],
    target_fields=["isbn", "title", "author", "genre", "price"],
)
```

- `get_records(sql)` — returns a list of tuples
- `get_df(sql)` — returns a pandas DataFrame
- `insert_rows(...)` — bulk insert with optional upsert support
- No need to manage connections manually — the hook handles open/close

Idempotency

Idempotency ensures that running a data pipeline multiple times with the same input produces the exact same output, preventing duplicates and data corruption

Exercise 1

After Exercise 1

We have a catalog in the database.
It loads the same ~~25~~ 24 rows whether you run it once or ten times.

Idempotency ✅

Stretch Goal

WHAT IF THE PUBLISHER SENDS A NEW CATALOG EVERY WEEK?

Each file may contain old books with updated prices and new books that were not there before.

```
books-01-05-2026.csv
books-07-05-2026.csv
books-14-05-2026.csv
```

ONE DAY, SOMEONE WIPES THE PROD DATABASE.

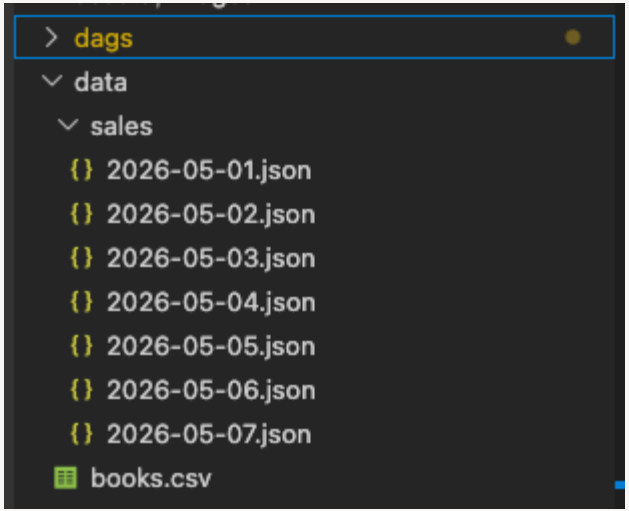
How do you recover? How do you ensure the same pipelines fetch the same data, in the right order, keeping the catalog up to date?

THAT IS OUR NEXT EXERCISE.

Module 2

Daily Sales Ingest

Scheduling in Airflow



Scheduling in Airflow

```python
@dag(
    schedule="@daily",        # run once per day
    start_date=datetime(2026, 5, 1),
    catchup=True,             # backfill all days since start_date
)
```

Common schedule values

- `None` — manual trigger only
- `"@daily"` — once per day at midnight
- `"@hourly"` — once per hour
- `"0 6 * * *"` — cron: 6am every day

Catchup

```
@dag(
    schedule="@daily",        # run once per day
    start_date=datetime(2026, 5, 1),
    catchup=True,             # backfill all days since start_date to today
)
```

1. Start date = 1 May 2026
2. Dag written and activated on - 13th May 2026
3. Schedule = Daily
4. No of catchup DagRuns = 13

> *if it's scheduled weekly?*

Catchup let's your DagRuns catchup(run) on all past schedules

Works only if `schedule ≠ None`

Logical Date

- The date the run *represents*, not when it actually ran.
- Eg., A dagrun scheduled at midnight might start running at `12:03`, but logical datetime will be `12:00`
- At a task level we can access them as parameters

```
@task
def load_file(ds=None):
    path = REPO_ROOT / "data" / "sales" / f"{ds}.json"
    #                                      ^ "2026-05-03" for the May 3 run
    return json.loads(path.read_text())
```

- At Operator level we can access them as jinja templates

```
echo_date = BashOperator(
  task_id="echo_date",
  bash_command="echo {{ ds }}",
)
```

Mini-Exercise

Explore other template fields

https://airflow.apache.org/docs/apache-airflow/stable/templates-ref.html

XComs

Passing Values Between Tasks

```
@task
def insert_sales(records, ds=None):
    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    rows = ...                  # Assume we fetched the rows here
    return len(rows)            # pushed to XCom automatically

@task
def log_summary(count, ds=None):
    print(f"Date: {ds} | Inserted: {count} records")
    #                               ^ pulled from XCom automatically & passed as
argument
```

- Return a value from `@task` and it is saved to XCom in the metadata DB
- Pass that return value as an argument to the next task — Airflow wires the pull automatically
- Keep XCom values small: strings, numbers, short lists. Not DataFrames or large files.

Exercise 2

Ingest Daily Sales

Load one JSON file per day using `ds`. Enable catchup and watch 7 backfill runs trigger.

`dags/02_daily_sales_starter.py`

After Exercise 2

Seven days of sales loaded automatically.
Each run knows its own date and loads only its own file.

Incremental Loading ✅
Backfill Data ✅
Scheduling Dags ✅

Stretch Goal

WHAT IF WE WANT TO VALIDATE THE DATA AS AND WHEN WE COMPLETE INGESTION?

How will we schedule the downstream dag based on upstream's completion

Assets

Problem: Time-based schedules don’t know if the upstream DAG finished. A DAG scheduled at 6am might start before yesterday’s data arrived.

Solution: Assets – a DAG declares what data it produces; downstream DAGs run when that data is ready, not on a clock.

Assets – Producer

```python
@task(outlets=[Asset("raw_sales")])
def insert_sales(ds=None, **context):
    # ... insert rows ...

    # Attach metadata to the asset event
    context["outlet_events"][Asset("raw_sales")].extra = {
        "date": ds,
        "count": len(records),
    }
```

- `outlets` declares what this task produces
- `outlet_events[asset].extra` attaches a dict that travels with the event

Assets – Consumer

```python
@dag(schedule=Asset("raw_sales"))   # fires when raw_sales is updated
def downstream():
    @task
    def print_event(**context):
        events = context["triggering_asset_events"][Asset("raw_sales")]
        print(events[0].extra)
        # {"date": "2026-05-01", "count": 5}
```

- `schedule=Asset( ... )` replaces a cron string – no fixed time needed
- `triggering_asset_events` gives the consumer access to the producer's extra data

Exercise 3a

Asset Handoff

Wire the producer → consumer handoff before Exercise 3b layers validation and HITL on top.

Module 3

Validate and Approve

Assets

SCHEDULES COUPLE PIPELINES BY TIME

DAG 04 running every hour does not know whether DAG 03 already finished. It just fires on the clock and hopes the data is ready.

ASSETS COUPLE PIPELINES BY DATA

An Asset is a named logical dataset. When a task emits an Asset, Airflow records it. Any DAG scheduled on that Asset wakes up automatically — no polling, no guessing.

THE RESULT

DAG 04 runs exactly when DAG 03 finishes and marks `daily_sales` ready — not a minute sooner, not a minute later.

Assets in Code

```python
from airflow.sdk import Asset

# Producer — emits the asset when the task completes
@task(outlets=[Asset("daily_sales")])
def load_sales( ... ):
    ...

# Consumer — wakes up when daily_sales is updated
@dag(schedule=Asset("daily_sales"))
def genre_report():
    ...
```

- `outlets` declares what data a task produces; Airflow records the update on success
- `schedule=Asset( ... )` replaces a cron string — the DAG runs on data, not on time
- The asset name is just a string; keep it stable across DAGs
- In this workshop: `sales_quarantine` and `daily_sales` are the two assets

Data Quality

BAD DATA IS WORSE THAN NO DATA

A negative quantity silently loaded into `daily_sales` skews every report downstream. By the time someone notices, the damage is done.

WHAT OUR SALES FILES CONTAIN

Each daily JSON has 10 records. Two are bad: one has `quantity: -2` and one references an ISBN not in the books catalog.

THE GOAL

Insert clean records. Quarantine bad ones with a reason. Then pause and ask a human — should we proceed?

validate_and_insert

```python
@task(outlets=Asset("sales_quarantine"))
def validate_and_insert(**context):
    events = context["triggering_asset_events"].get(Asset("raw_sales"), [])
    ds = events[0].extra["date"]
    records = json.loads((REPO_ROOT / "data" / "sales" / f"{ds}.json").read_text())

    hook = PostgresHook(postgres_conn_id="bookshop_postgres")
    known_isbns = {row[0] for row in hook.get_records("SELECT isbn FROM books")}

    valid_rows, bad_rows = [], []
    for rec in records:
        if rec["quantity"] <= 0:
            bad_rows.append((json.dumps(rec), f"invalid quantity:
{rec['quantity']}"))
        elif rec["isbn"] not in known_isbns:
            bad_rows.append((json.dumps(rec), f"unknown isbn: {rec['isbn']}"))
        else:
            valid_rows.append((rec["isbn"], ds, rec["quantity"], rec["total"]))

    hook.insert_rows("daily_sales", valid_rows, ...)
    hook.insert_rows("sales_quarantine", bad_rows, ...)
    return markdown_table(bad_rows)   # returned as XCom
```

One loop. Valid rows go to `daily_sales`. Bad rows go to `sales_quarantine`. The return value is a formatted table — XCom will carry it to the next task.

Human-in-the-Loop

```python
from airflow.providers.standard.operators.hitl import ApprovalOperator

approve = ApprovalOperator(
    task_id="approve_or_reject",
    subject="Quarantined sales records require your review",
    body="""
Date: {{ ds }}

{{ ti.xcom_pull(task_ids='validate_and_insert') }}

Approve to continue. Reject to stop this run.
    """,
    outlets=[Asset("daily_sales")],
    response_timeout=timedelta(hours=24),
)
```

WHAT HAPPENS IN THE UI

Airflow pauses the DAG and shows the quarantine table — pulled from XCom — inside an Approve/Reject form. No external tools needed.

ON APPROVE

The `daily_sales` asset is emitted and DAG 04 triggers automatically.

Exercise 3b

Validate Sales + Human-in-the-Loop

Split valid and bad records, quarantine the bad ones, then approve in the Airflow UI.

`dags/03b_validate_sales_starter.py`

After Exercise 3b

Valid records are in `daily_sales`. Bad records are in quarantine.
A human reviewed and approved the data directly in the Airflow UI before the pipeline continued.

✅ Assets ✅ Human in the loop

Module 4

Genre Report

Dynamic Task Mapping

THE PROBLEM WITH A FOR-LOOP

If you aggregate each genre inside a Python loop, all 5 genres run sequentially in one task. You cannot retry a single genre, and the UI shows one opaque task.

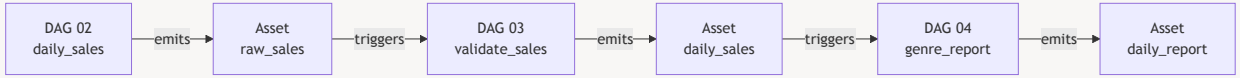
THE BETTER WAY: .EXPAND()

Tell Airflow to run `build_genre_report` once per genre. Airflow creates one task instance per value at runtime. They run in parallel, and each has its own log and retry.

```
genres = get_genres()                          # returns ["Fiction", "Mystery", ...]
genre_rows = build_genre_report.expand(genre=genres)
#                                              ^ one task instance per genre
```

The number of mapped tasks is only known at runtime. Airflow shows them as `build_genre_report[0]`, `build_genre_report[1]`, etc. in the UI.

Assets Recap — The Full Pipeline



- Every downstream DAG wakes up on data, not on a clock
- Open Airflow UI **Assets** tab to see this graph live
- If any DAG fails, nothing downstream runs on stale data
- Each DAG can be maintained, retried, and tested independently

Consuming an Asset

```python
# DAG 03 emits daily_sales when the human approves
approve = ApprovalOperator(
    task_id="approve_or_reject",
    outlets=[Asset("daily_sales")],
    ...
)

# DAG 04 wakes up the moment daily_sales is updated
@dag(
    dag_id="04_genre_report",
    schedule=Asset("daily_sales"),
    catchup=False,
)
def genre_report(): ...
```

- `schedule=Asset( ... )` replaces a cron string entirely
- `catchup=False` is correct here — the asset event itself carries the context
- The run is linked to the asset update in the UI for full traceability

Exercise 4

Build the Genre Report

Aggregate sales per genre using `.expand()`, then watch DAG 04 trigger automatically when DAG 03 approves.

`dags/04_genre_report_starter.py`

After Exercise 4

**Five genre tasks ran in parallel.
DAG 04 started automatically the
moment DAG 03 finished loading data.**

The full pipeline now runs end to end, triggered by data, not by clocks.

From Fragile Scripts

to a Trusted Pipeline

RELIABLE INGEST

Books catalog loads idempotently — same result every run

INCREMENTAL LOADS

Each sales file loaded for its own date, backfillable on demand

QUALITY GATES

Bad records quarantined with a reason, never silently loaded

EVENT-DRIVEN REPORTING

Genre report builds automatically when validated data is ready

What You Learned

- DAG anatomy: `@dag` , `@task` , `>>` , manual trigger, Airflow UI
- Connections and PostgresHook — credentials out of code
- Scheduling, catchup, Jinja `{{ ds }}` , logical date, XCom
- Branching with `@task.branch` , trigger rules, quarantine pattern
- Dynamic Task Mapping with `.expand()` for parallel processing
- Assets — event-driven scheduling between decoupled DAGs

Explore Next

Go deeper on Airflow

- **TaskGroups** — visually group related tasks inside a DAG
- **Sensors** — wait for a file, a table, or an external condition
- **Providers** — installable packages for S3, GCS, Slack, dbt, and more
- **KubernetesExecutor** — run each task in its own pod at scale
- **Deferrable operators** — async waiting without blocking a worker slot

Grow the system

- dbt for SQL transform ownership
- Great Expectations for richer data quality contracts
- Asset-aware orchestration with multiple upstream dependencies
- Airflow Variables and Connections for environment-based config
- AI-aware operators and service integrations

Thank You

Questions, ideas, and extensions are welcome.

https://www.linkedin.com/in/bhavanicodes
bhavanicodes@gmail.com

Appendix — Hosting Options

Option	Best for	Tradeoff
`airflow standalone`	Local dev, workshops	Single process, not for production
Docker Compose	Team dev, CI	Easy setup, executor limited to Local
Self-hosted Kubernetes	Full control, large scale	High ops overhead
Amazon MWAA	AWS-native shops	Managed, but version lag and limited config
Google Cloud Composer	GCP-native shops	Managed, expensive at small scale
Astronomer	Enterprise, any cloud	Fully managed + support, paid